SnapStak Studio

Complete Engineering Reference

An open-source VS Code extension, in active development, for the community to build on

A full code breakdown for a developer joining the project: how to open and build it,
how the integrated browser and AI tools work, and how every panel and view fits together.
Unzip an AI-generated project, run it in an embedded browser, and refine it - inside VS Code.

Russel Hawkins | SnapStak.ai

Contents

1. Introduction

1.1 What this software does

SnapStak Studio is an open-source Visual Studio Code extension. Its purpose is to be the place where a project produced by the SnapStak / CON10X engine is brought into the editor, run, and refined - without leaving VS Code. The working core today is exactly that: a developer takes an AI-generated project as a zip, Studio unzips it, detects its framework, runs it, and shows it live in an embedded browser. Around that core sits a set of AI-assisted tools - a copywriter, a database designer, design canvases, animation and media helpers - most of which are functional. The published name in the marketplace is "SnapStak.ai"; this document refers to the product as SnapStak Studio throughout.

Project status: Studio is a work in progress, published open source so that other developers can extend it. It is not a finished product, and not every tool is complete. The reliable, working path is import-and-run-in-the-embedded-browser; the AI panel tools range from fully functional to partial. This document describes the codebase as it stands and flags where a capability is a foundation to build on rather than a finished feature, so a contributor knows where the solid ground is and where the open work lies.

1.2 Where Studio sits in the SnapStak suite

Studio is the third tool in the SnapStak suite, alongside SnapStak Desktop (the Chrome extension and CON10X Web Domain engine) and SnapStak Mobile (the MAUI Android app). Desktop and Mobile capture an interface and generate code; Studio is intended to be where that code is then run and refined. Critically, Studio does not contain the CON10X engine - the engine stays on the SnapStak server. Studio is a client: it authenticates to that server and calls its APIs (for example the copywriter generation endpoints). This separation is deliberate and is why Studio can be open source while the engine remains proprietary - the value the subscription unlocks is the engine, and Studio is the free, inspectable front end to it.

1.3 How it is built: a VS Code extension

Studio is written in TypeScript against the VS Code Extension API. It is not a web app and not a desktop app - it runs inside VS Code as an extension, contributing an activity-bar container, several sidebar views, a secondary-sidebar AI panel, and a number of full-editor webview panels. Everything the user sees is either a native VS Code tree view or an HTML webview the extension renders and drives by message passing. Understanding that webview model (covered in section 3) is the single most useful thing before reading any panel file.

1.4 Who this document is for

This is for a developer who has just been handed the repository. It assumes you can read TypeScript and have seen the VS Code extension model at least once, but it assumes nothing about this project. It begins with opening and building the extension, then explains the architecture, then walks each capability in detail, and ends with reference appendices.

2. Opening and Building the Project

The repository is a standard VS Code extension project: a TypeScript source tree, a `package.json` manifest, and a `tsconfig.json`. You open the folder itself in VS Code (not a .sln - there isn't one) and build with the TypeScript compiler.

2.1 Prerequisites

- **Visual Studio Code** 1.85 or later - the manifest sets `"engines": { "vscode": "^1.85.0" }`.
- **Node.js** (version 20 is the type-target; `@types/node` is ^20). Node is needed for npm and the TypeScript build.
- **TypeScript 5.3+** and the dev dependencies in the manifest (the TypeScript ESLint plugin and parser, ESLint). These install with `npm install`.
- **A running SnapStak server** for the authenticated features. By default Studio talks to `http://localhost:3001` (configurable). Without it, project import, running, and the design canvases still work, but sign-in and the AI copywriter calls will not.

2.2 Project structure

Path	Contents
`src/extension.ts`	The entry point: activate() wires up every view, panel, and command.
`src/snapstakClient.ts`	The CON10X server client: HMAC sign-in, the session token, and all server API calls.
`src/devRunner.ts`	Framework detection, the dev server launcher, and the integrated browser webview.
`src/views/`	The sidebar tree views and webview view providers (workspaces, local projects, settings, auth, the AI panel, database schema).
`src/panels/`	The full-editor webview panels: the eight AI tabs plus the drawing, database, and copywriter canvases.
`src/copywriter/`	The copywriter back end: segment registry, segment injector (writes edits back to source), and panel state.
`media/`	The extension icon and SVG assets.
`out/`	Compiled JavaScript output from tsc (generated; not edited by hand).
`docs/`	A setup guide PDF.
`package.json` / `tsconfig.json`	The extension manifest and the TypeScript build configuration.

2.3 The manifest: package.json

The `contributes` section of package.json is the map of everything Studio adds to VS Code. Reading it tells you the whole surface area before you read a line of logic.

Contribution	What it declares
viewsContainers	An activity-bar container (snapstak) and a secondary-sidebar container (snapstak-panel).

Contribution	What it declares
views	In the activity bar: Workspace (file tree), Local Projects (zip list), and a Settings webview. In the secondary sidebar: the SnapStak AI webview panel.
commands	Seven commands: extract/open/run a project, refresh the views, and open the dashboard (covered in Appendix A).
menus	Where the commands appear - title-bar buttons and right-click context actions on tree items.
configuration	One setting, snapstak.serverUrl, defaulting to http://localhost:3001.

Note `"activationEvents": ["*"]` - Studio activates as soon as VS Code finishes starting, so its panels are present immediately rather than waiting for a command. The `main` entry points at the compiled `./out/extension.js`.

2.4 TypeScript configuration and the build

tsconfig compiles `src/` to `out/` as CommonJS targeting ES2020, with strict type-checking and source maps on. The npm scripts are straightforward: `npm run compile` does a one-shot build (`tsc -p ./`), `npm run watch` rebuilds on save, and `npm run lint` runs ESLint over the source. The `vscode:prepublish` hook runs compile automatically when the extension is packaged.

2.5 Running the extension in development

1. **Open the repository folder in VS Code** and run `npm install` once to restore dependencies.
2. **Press F5** (Run Extension). VS Code compiles the project and launches a second "Extension Development Host" window with Studio loaded.
3. **In that host window,** the SnapStak icon appears in the activity bar. Open it to see the Workspace, Local Projects and Settings views; the AI panel appears in the secondary sidebar.
4. **Point a workspace folder** at a directory of project zips via Settings, then extract and run a project. For the authenticated AI features, sign in with a SnapStak API key (the server must be reachable).

On dependencies: install npm packages with `npm install <pkg>` or through the editor; the project intentionally keeps dependencies minimal - only the VS Code API and the TypeScript toolchain - so the extension stays light and auditable.

3. Architecture Overview

3.1 The extension host and the webview model

A VS Code extension runs in the extension host - a Node.js process, separate from the editor UI. The extension cannot draw arbitrary UI directly; it works in two ways. Native tree views (the Workspace and Local Projects lists) are described to VS Code through a data provider and rendered by the editor. Richer UI is built as a webview - a sandboxed HTML page the extension supplies and communicates with only by passing messages. Studio uses both: tree views for file lists, and webviews for everything visual (the AI panel, the browser, the canvases, the copywriter).

Why this matters for every panel file: the panel files are mostly HTML/CSS/JS strings that get loaded into a webview, plus a message handler on the extension side. The webview and the extension are different worlds - the webview cannot touch the file system or call the server directly. It posts a message; the extension does the privileged work and posts a result back. When you read a panel, you are reading two halves: the UI (in the webview HTML) and the controller (the onDidReceiveMessage handler in the extension).

3.2 The activation sequence

activate() in `extension.ts` runs once on startup and is the spine of the app. In order, it:

5. Creates the SnapStakClient (the server client) and loads any stored session token.
6. Initialises the copywriter with the context and client so it can auto-open when caption blocks arrive from the browser.
7. Wires the browser webview's messages to the copywriter (selection-complete events become copywriter LOAD_BLOCKS messages).
8. Registers the secondary-sidebar AI panel, the Workspace tree, the Local Projects tree, the Settings webview, and a file-decoration provider for colouring zip items.
9. Registers the seven commands (extract, open, refresh views, run project, open dashboard) and a few internal commands (open schema, open drawing canvas, open copywriter).
10. Auto-opens both the AI panel and the SnapStak activity-bar container so the tools are visible immediately.

3.3 The shared browser webview

A central design decision: there is one browser webview, shared. The Media, Copywriter and Database tabs of the AI panel all use that same live browser as their interaction surface, rather than each opening its own. `devRunner.ts` owns it and exposes `postToBrowser()` to send it messages and `setBrowserMessageHandler()` to receive them. The browser sits idle until a tool wakes it, and tools coordinate through it. This is why so many files import from devRunner - it is the hub the visual tools talk through.

3.4 The end-to-end workflow

This is the workflow Studio is designed around. Steps 1 to 3 - sign in, import, and run in the embedded browser - are the solid, working core. Step 4 covers the AI tools, which vary in

completeness; step 5 (write-back, and ultimately deploy) is partly built and is a natural area for contributors.

11. **Sign in.** The user enters a SnapStak API key; SnapStakClient signs in over HMAC and stores a session token.
12. **Import.** The user points Studio at a folder of project zips. Clicking a zip extracts it, runs npm install, and creates a matching workspace on the server.
13. **Run.** Clicking the project (or its index.html) detects the framework, starts the right dev server, waits for the port, and opens the integrated browser at the running app.
14. **Edit with AI.** The user works through the AI panel tabs - rewriting copy with the Copywriter, designing a schema with the Database tab, drawing with the Canvas, and so on - many of these interacting live with the running app in the browser.
15. **Write back and refine.** Tools that change content (notably the Copywriter) write their changes back into the project's source files, and the developer continues refining from there. (A one-click deploy is part of the product vision but is not a completed feature in this build.)

4. The Server Client and Authentication

Location: `src/snapstakClient.ts`. This single class is Studio's entire relationship with the SnapStak server. Because the CON10X engine lives server-side, every AI capability ultimately routes through here.

4.1 SnapStakClient and HMAC sign-in

Authentication is a two-step design that trades a slow signed handshake for fast token calls. The user provides an API key. `authenticate()` builds a set of security headers - the machine id, a timestamp, and a random nonce - and signs the string `machineId:timestamp:nonce` with HMAC-SHA256 keyed by the API key. It posts those to `/api/plugin/auth` and receives an 8-hour session JWT in return. The API key and token are then stored, and the real user name is fetched from `/api/plugin/me` (the auth response never includes it, and the name matters for billing and workspace records).

4.2 The session lifecycle and SecretStorage

Every subsequent call goes through a private `request()` helper that sends the session JWT as a bearer token - no re-signing, so calls are fast. If a call returns 401 (token expired), the client transparently re-authenticates with the stored API key and retries once; if that fails, the user is asked to re-enter the key. All secrets - the API key, the session token, the user id, the workspace name, the user's name and email - are kept in VS Code's `SecretStorage`, the OS-backed encrypted store, never in plain settings or files. Signing out best-effort deregisters on the server and then clears local secrets.

4.3 Workspaces and the CON10X server API

Opening a project creates a server-side workspace named after the zip (sanitised to a safe slug). The client also exposes methods to list and fetch workspaces, and - the heart of the AI features - the copywriter calls: generate a business description from keywords, generate one heading or all headings for the captured blocks, and rewrite body text to a target word count. Each of these posts to a `/api/copywriter/*` endpoint with the bearer token, so the engine work happens on the server and Studio just sends inputs and renders results.

5. Projects: Import, Run, and the Live Browser

5.1 The Local Projects and Workspace views

Two sidebar tree views manage projects. `ZipViewProvider` (Local Projects) lists every .zip in the configured folder, showing an orange zip icon for unextracted archives and a green folder icon for ones already extracted. Clicking an unextracted zip extracts it and runs npm install; clicking an extracted one loads its file tree. `WorkspacesViewProvider` (Workspace) then shows that project's file tree, and clicking its index.html opens the shared browser. The two views are linked - the zip view drives what the workspace view shows.

5.2 devRunner - framework detection and dev server

Location: `src/devRunner.ts`. When the user runs a project, `detectFramework()` reads its package.json and maps its dependencies to the right dev command and port - Next.js, Angular, Vue CLI, Nuxt, Vite, React (react-scripts), SvelteKit, Remix, or a static fallback, each with its known default port. `launchDevServer()` then runs npm install if node_modules is missing, starts the dev command in a terminal, and `waitForServer()` polls the port until the server answers (up to a timeout) before opening the browser. This is what makes "click a project, see it running" a single action.

Detected framework	Command	Port
Next.js	`npm run dev`	3000
Angular	`npm run start`	4200
Vue CLI	`npm run serve`	8080
Nuxt	`npm run dev`	3000
Vite	`npm run dev`	5173
React (react-scripts)	`npm start`	3000
SvelteKit	`npm run dev`	5173
Remix	`npm run dev`	3000
Static / unknown	(serve or generic dev)	0 / 3000

5.3 The integrated browser and ss-tracker injection

The integrated browser is a webview pointed at the running dev server. To let the visual tools interact with the live page, devRunner injects a small tracker script: before launch it copies `ss-tracker.js` into the project root and appends a `<script>` tag to index.html (marked with a comment so it can be cleanly removed). The script loads with the page and stays idle until a tool wakes it via `postToBrowser({ command: 'startTracker' })`. When the browser panel closes, both the tag and the copied file are removed, leaving the project clean. This injection - and its tidy removal - is what lets the Copywriter and other tools select and act on real elements in the running app.

6. The Eight-Tab AI Panel

6.1 The panel provider and tab model

Location: `src/views/snapstakPanelProvider.ts`. The secondary-sidebar panel is the main AI surface - a single webview hosting eight tabs, each built by its own panel module and concatenated into the panel HTML. Three of the tabs (Media, Copywriter, Database) are "browser tabs": selecting them reveals the shared browser webview, because they operate on the live running page. The others are self-contained. Switching tabs is handled in the webview; the heavier tools open as full-editor panels when needed.

6.2 The capability tabs

Tab	Module	Purpose
Media	`mediaPanel.ts`	Agentic imaging - working with images against the live page (a browser tab).
Copywriter	`copywriterPanelTab.ts` / `copywriterPanel.ts`	AI copywriting over the page's text blocks (a browser tab; opens a full panel). See section 7.
Actions	`actionsPanel.ts`	Action helpers within the panel.
Form	`formPanel.ts`	Form building.
Animation	`animationPanel.ts`	An animation engine - configure animation type and settings for elements.
Database	`databasePanel.ts`	Entry to the database designer (a browser tab; opens the schema canvas). See section 8.
Canvas	`canvasPanel.ts`	A vector drawing canvas with select/node/shape tools.
Library	`libraryPanel.ts`	A library surface (stub in this build).

A note on scope: the panel modules are mostly large inlined HTML/CSS/JS strings - the UI for each tab - returned by a `get<Name>Panel()` function. The genuinely heavy tools (Copywriter, Database, the drawing canvas) have a thin tab in the panel that opens a separate full-editor webview panel where the real work happens; those are the next two sections.

7. The Copywriter

The Copywriter is the most developed AI tool in Studio and the clearest example of the full loop: select real text in the running app, rewrite it with AI, and write the result back into the source files. It spans `panels/copywriterPanel.ts` (the full-editor panel), `panels/copywriterPanelTab.ts` (the sidebar tab), and the `src/copywriter/` back end.

7.1 The capture-to-rewrite flow

When the user selects content in the live browser, the tracker sends a `COPYWRITER_SELECTION_COMPLETE` message. extension.ts forwards it to the copywriter as a `LOAD_BLOCKS` message - and crucially this works whether or not the copywriter panel is open yet: if it is closed, the message is held and replayed once the panel's webview signals it is ready. The selected content arrives structured as blocks, each with a heading (H1/H2/H3 with a word count) and arrays of body text, buttons and links.

7.2 The six-phase panel and AI calls

The panel models the rewrite as a six-phase workflow (the `Phase` type in `copywriter/state.ts` runs 1 through 6): gather a business description and keywords, generate and confirm headings, then rewrite body text block by block to a target word count, tracking each block as untouched, partial, or complete. The actual generation calls go through SnapStakClient to the server: `generate-description`, `generate-heading` / `generate-headings`, and `rewrite-body` (which returns the new text, its word count, and a needs-review flag when it misses the target). The panel holds all of this in a single `PanelState` object so the workflow can be resumed.

7.3 Segments and writing back to source

The hard part of any "edit the live page, change my code" tool is mapping a rendered element back to the exact place in the source that produced it. Studio solves this with segments. `copywriter/segmentRegistry.ts` maintains `.snapstak/segments.json` in the project, assigning each element a globally-unique 8-character hex id tied to its structural DOM path, source file, and tag. `copywriter/segmentInjector.ts` then locates each element in the JSX/HTML source by matching opening tags, using a framework-aware attribute map (so it knows React uses className and htmlFor where HTML uses class and for, and similarly for Next.js and others) and scanning only the source folders (src/components, src/pages, src/app). That mapping is what lets an AI rewrite in the browser become a precise edit to the right line of the right component file.

8. The Database Designer

8.1 The drag-and-drop schema canvas

Location: `panels/databaseCanvasPanel.ts` (the full-editor canvas) with `views/databaseSchemaViewProvider.ts` mirroring a compact version in the sidebar. The Database tab opens a drag-and-drop schema builder where the user lays out databases, tables and enums as blocks, defines columns, and draws relationships - foreign-key columns connect to the primary-key dots of referenced tables, and enum columns connect to their enum blocks, with connector lines drawn live between them. It is, in effect, an ER diagram editor built as a webview.

8.2 schema.json and multi-engine SQL export

The canvas persists to `schema.json` in the project's schema folder (a CMS-friendly format), auto-saving as the user edits. From that single schema it exports engine-specific SQL: the export writes a `schema.<engine>.sql` file, with the engine chosen by the user - the canvas supports the major SQL and document engines (PostgreSQL, MySQL, MariaDB, SQLite, MSSQL/SQL Server, Oracle, CockroachDB, MongoDB among them). Because the design is captured once as schema.json and rendered to whichever dialect is requested, the same diagram produces correct DDL for any supported engine - the same "design once, target anything" idea that runs through the whole SnapStak suite.

9. The Design Canvases

Two further full-editor webview panels round out the visual tooling. `panels/drawingCanvasPanel.ts` is a rectangle-and-shape drawing editor with select, resize, move, and image/icon placement - a lightweight design surface inside the editor. `panels/canvasPanel.ts` provides the vector Canvas tab with select and node-editing tools for working with paths. Like every other webview, both are HTML/CSS/JS rendered by the extension and driven by message passing; the extension side handles file dialogs and persistence, while the webview handles the interactive drawing. `panels/animationPanel.ts` similarly provides an animation configuration engine within the AI panel.

10. Getting Started as a Developer

10.1 Suggested reading order

Read the code in this order rather than alphabetically:

16. `package.json` (the contributes section) - the whole surface area of the extension.
17. `src/extension.ts` - activate() ties everything together; it is the map.
18. `src/snapstakClient.ts` - how Studio authenticates and talks to the engine.
19. `src/devRunner.ts` - framework detection, the dev server, and the shared browser.
20. `src/views/snapstakPanelProvider.ts` - the eight-tab panel and the browser-tab model.
21. `src/copywriter/` + `panels/copywriterPanel.ts` - the fullest example of the capture-edit-writeback loop.
22. `panels/databaseCanvasPanel.ts` - the schema designer and SQL export.

10.2 Mental models to keep

- **Studio is a client, not the engine.** The CON10X engine is on the server. Studio authenticates and calls APIs. This is why it can be open source while the engine stays proprietary.
- **Two worlds: extension host and webview.** Every panel is UI in a webview plus a controller in the extension, communicating only by messages. The webview cannot touch files or the network directly.
- **One shared browser.** The Media, Copywriter and Database tools all act through a single shared browser webview owned by devRunner - not their own browsers.
- **Segments are how edits reach source.** A rendered element maps back to a source line through the segment registry and the framework-aware injector. That mapping is the core of "edit live, change code".

10.3 Common gotchas

- **Sign-in fails or AI calls error** - the SnapStak server must be reachable at snapstak.serverUrl (default http://localhost:3001), and the session JWT expires after 8 hours (the client auto-retries once on 401).
- **A tool does nothing on the page** - the shared browser must be open and the ss-tracker script must have been injected and woken via startTracker. If the browser was never launched, there is nothing to act on.
- **Copywriter edits land in the wrong place** - check the segment registry and that the source lives under src/components, src/pages or src/app, since the injector only scans those.
- **Leftover ss-tracker tag** - it is removed when the browser panel closes; an abnormal exit can leave it, marked by its snapstak-tracker comment, safe to delete.
- **Dependencies** - keep them minimal and install via npm; the extension deliberately depends only on the VS Code API and the TypeScript toolchain.

Appendix A: Commands, Views, and Configuration

A.1 Commands

Command id	Title	Action
snapstak.extractZip	Extract & Open Project	Extract a selected zip and npm install.
snapstak.openExtracted	Browse Project	Load an already-extracted project's file tree.
snapstak.refreshZipView	Refresh Local Projects	Re-scan the workspace folder for zips.
snapstak.refreshWorkspaces	Refresh Workspace	Refresh the active project file tree.
snapstak.runProject	Run Project	Detect framework, start dev server, open the browser.
snapstak.openDashboard	Open Dashboard	Open snapstak.ai in the external browser.

A.2 Views

View id	Container	Kind
snapstak.workspacesView	Activity bar (snapstak)	Tree - the active project's files.
snapstak.zipView	Activity bar (snapstak)	Tree - local project zips.
snapstak.settingsView	Activity bar (snapstak)	Webview - workspace folder settings.
snapstak.panelView	Secondary sidebar	Webview - the eight-tab AI panel.

A.3 Configuration

One setting: `snapstak.serverUrl` (string, default `http://localhost:3001`) - the SnapStak server the client authenticates and makes API calls against.

Appendix B: The Source Layout

B.1 Top-level modules

File	Role
extension.ts	Activation: registers views, panels, commands; wires the browser to the copywriter.
snapstakClient.ts	Server client: HMAC auth, session JWT, workspace and copywriter API calls.
devRunner.ts	Framework detection, dev server launch, the shared browser webview and tracker injection.

B.2 views/

File	Role
snapstakPanelProvider.ts	The eight-tab AI panel webview.
workspacesViewProvider.ts	The active project's file tree.
zipViewProvider.ts	The local project zip list, extract/open actions, and label colouring.
settingsViewProvider.ts	The workspace folder settings webview.
authViewProvider.ts	The API key entry panel.
databaseSchemaViewProvider.ts / databasePrefsViewProvider.ts	The sidebar schema canvas and its preferences.

B.3 panels/

File	Role
copywriterPanel.ts / copywriterPanelTab.ts	The copywriter full panel and its sidebar tab.
databaseCanvasPanel.ts / databasePanel.ts	The schema canvas and the database tab.
drawingCanvasPanel.ts / canvasPanel.ts	The drawing editor and the vector canvas tab.
animationPanel.ts / mediaPanel.ts / formPanel.ts / actionsPanel.ts / libraryPanel.ts	The remaining AI panel tabs.

B.4 copywriter/

File	Role
segmentRegistry.ts	Maintains .snapstak/segments.json - unique ids tying elements to source.
segmentInjector.ts	Locates and edits elements in source files, framework-attribute aware.
injection.ts / state.ts	Injection helpers and the six-phase copywriter panel state.

This document describes the SnapStak Studio VS Code extension. The source is at github.com/SnapStak-AI/SnapStak-Studio and is published under the MIT License. Contributions and derivative works are welcome under the terms of that licence.